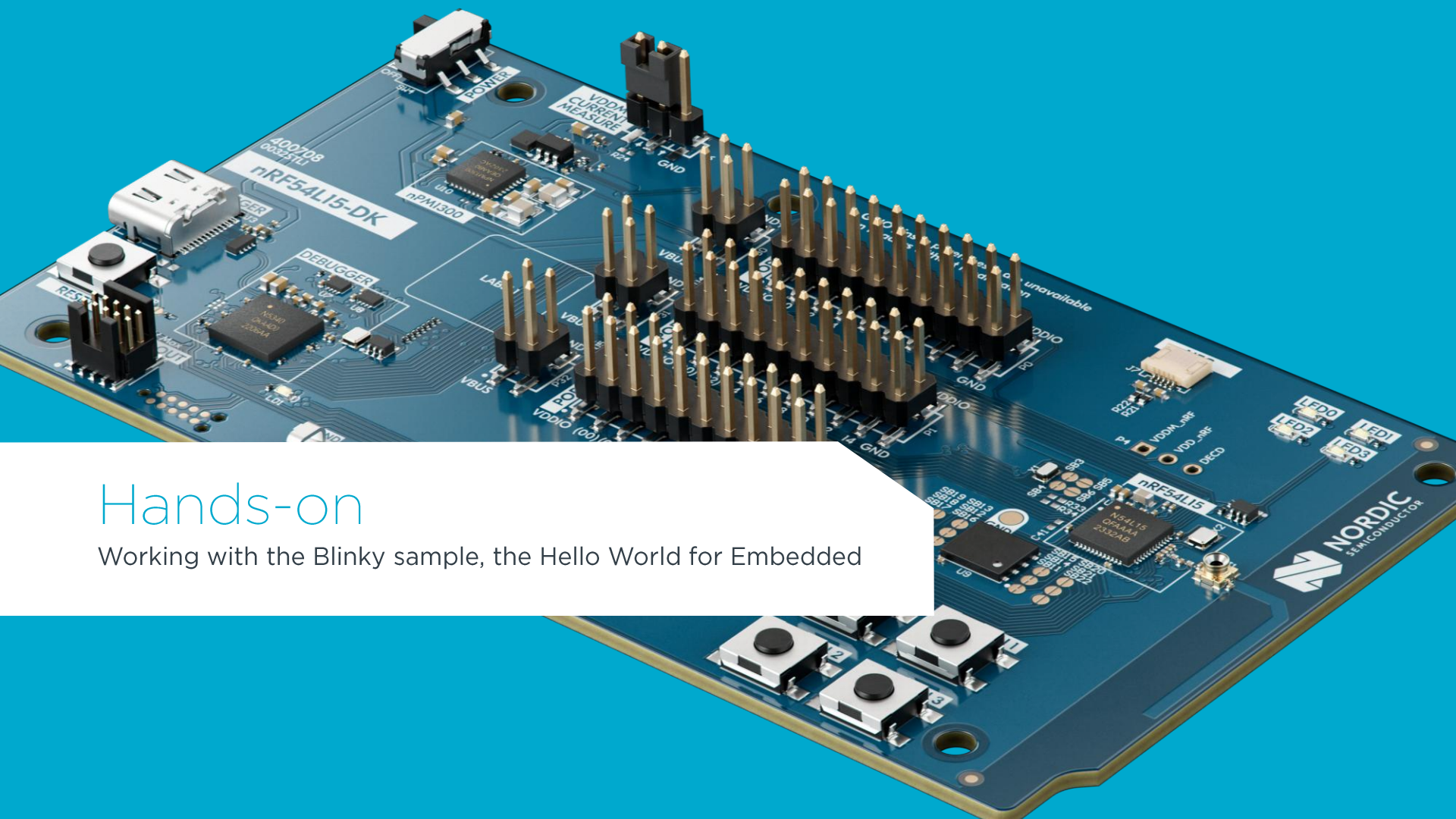


Hands-on

Working with the Blinky sample, the Hello World for Embedded

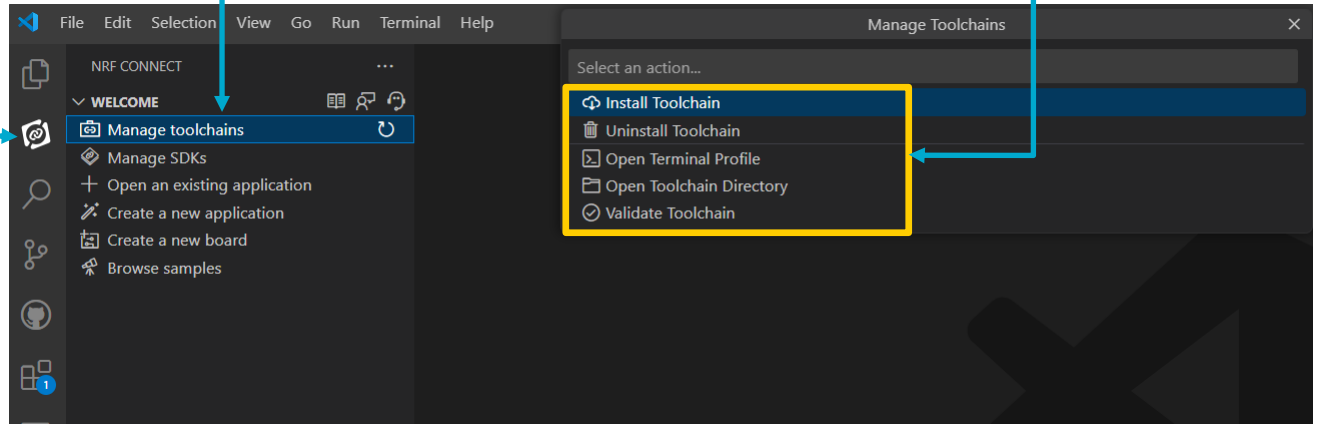
© Nordic Semiconductor

Manage Toolchain

Select the nRF Connect Extension

Manage toolchains

Install Toolchain, default installation path
<drive>:\ncs\toolchain\<build version>



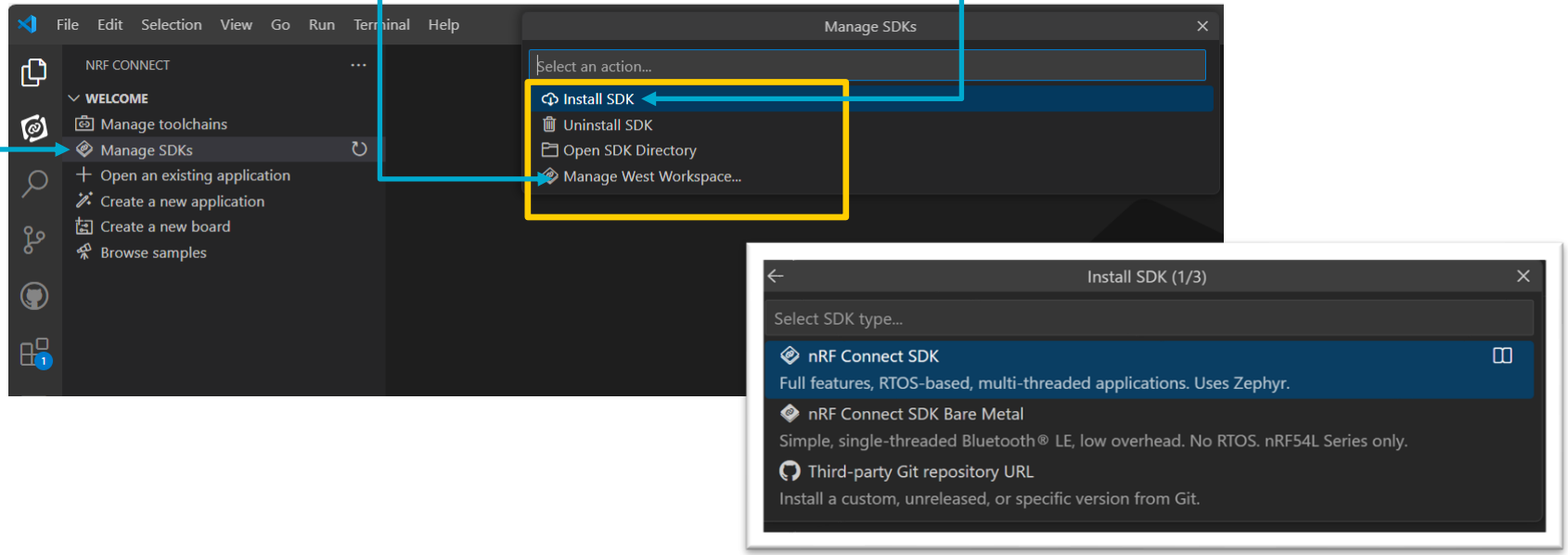
© Nordic Semiconductor

Manage SDK's installations

Manage SDKs

Create/open
Workspace projects

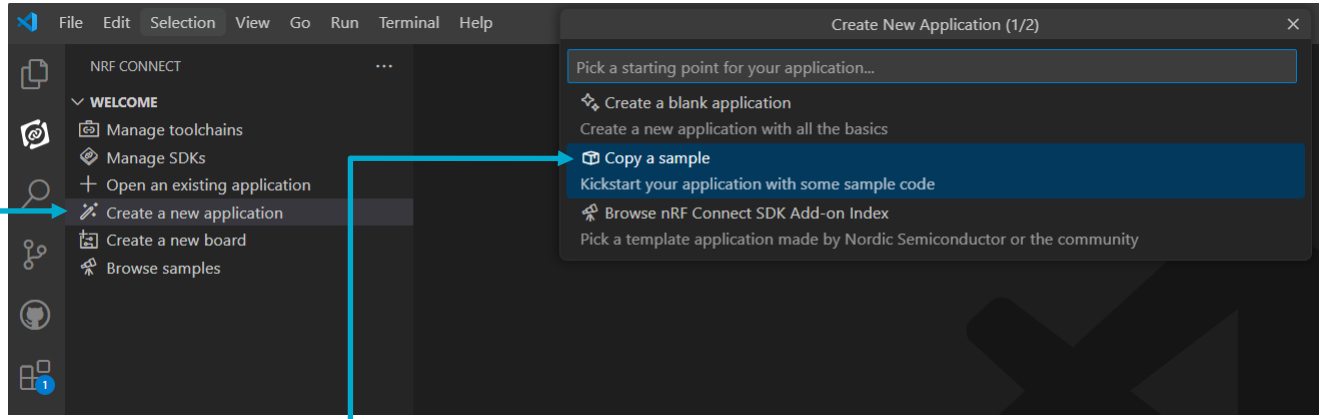
Install SDK, default installation path
<drive>:\ncs\v3.2.1



Building *blinky* Project

Building a application

Create a new application

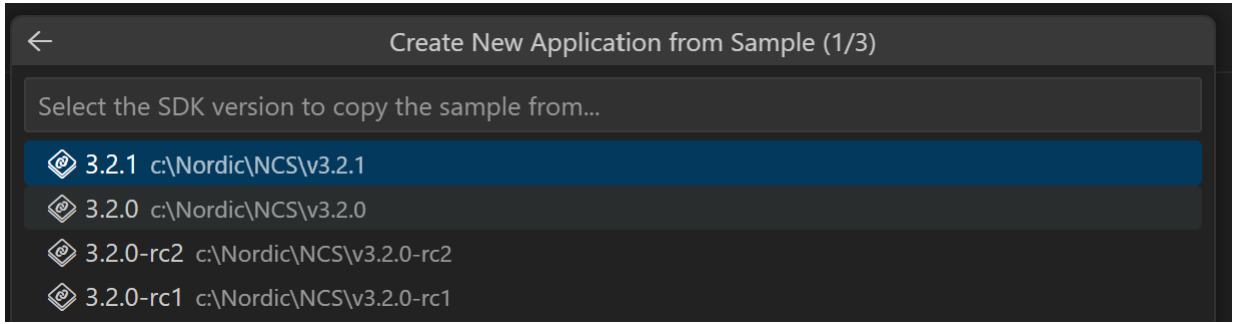


Copy a sample

This will create a copy from a sample or application out of the SDK repository. This will create a stand-alone application.

© Nordic Semiconductor

Select the version of the SDK



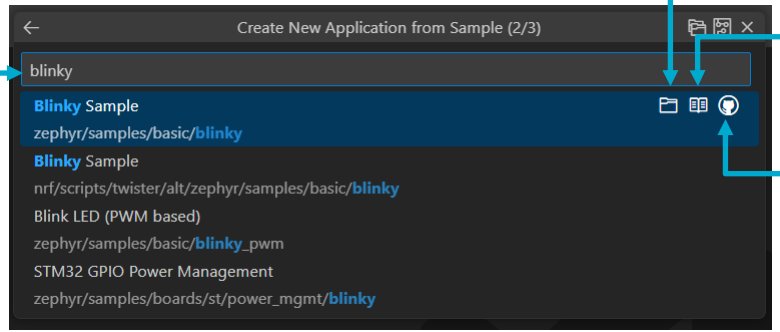
Copy a sample

This will ask you to select a SDK version to browse and copy the samples or application.

© Nordic Semiconductor

Selecting a sample: blinky

Search for the **“blinky”** sample that is part of the **zephyr** repository



Open file location

Open documentation

Open github repository

If the location of the sample/application starts with **“zephyr”** this sample is provided by Zephyr.

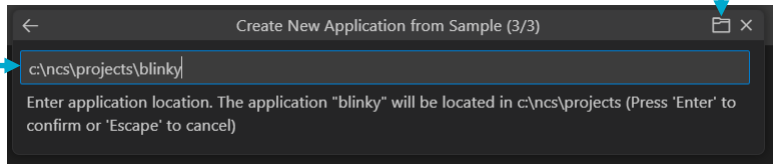
If the location starts with **“nrf”** the sample is provided by Nordic Semiconductor and part nRF Connect SDK

© Nordic Semiconductor

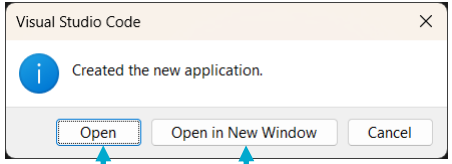
Location of new application

Full path to location of project.
By default the last used location followed by the **application name.**

Use icon to browse for target file location of new application

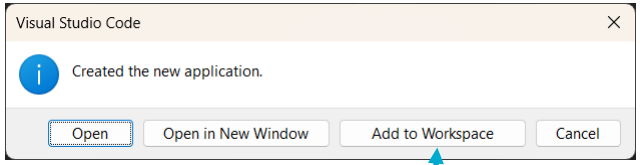


Where to open the new application



Open project in current Visual Code instance

Open a new Visual Code instance and open the project

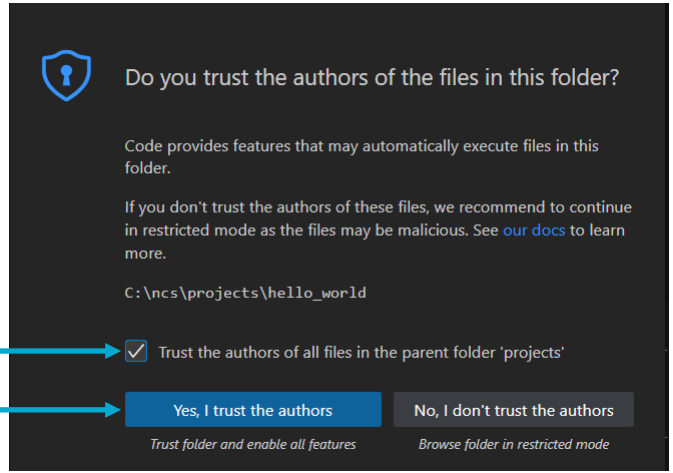


Add project to workspace in current Visual Code instance

Permission to access project location

Optional to trust the complete parent folder of the project

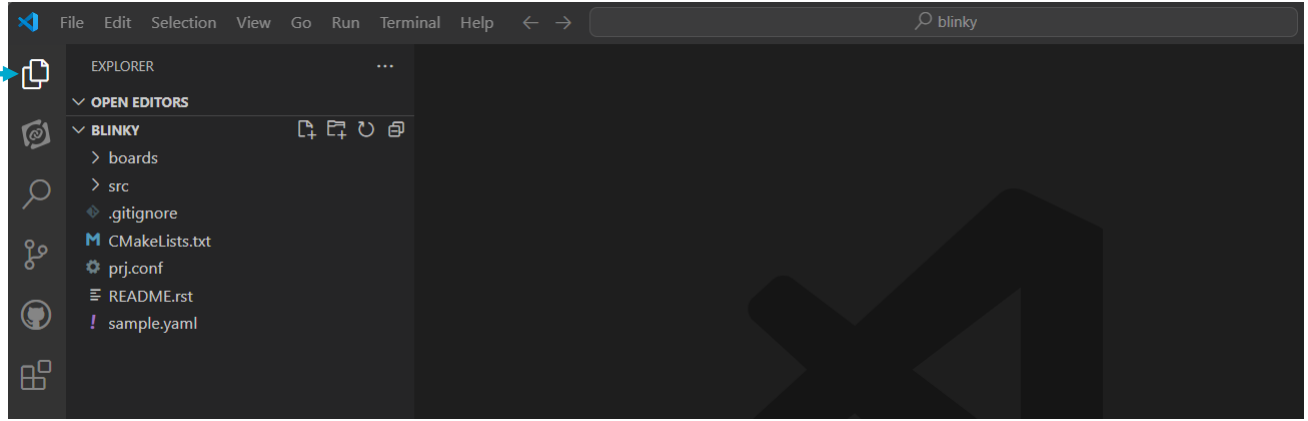
Mandatory for Visual Code to trust the location of the project



Application created

The new application based on the sample has been created in a Visual Code Workspace (not to confuse with West Workspace)

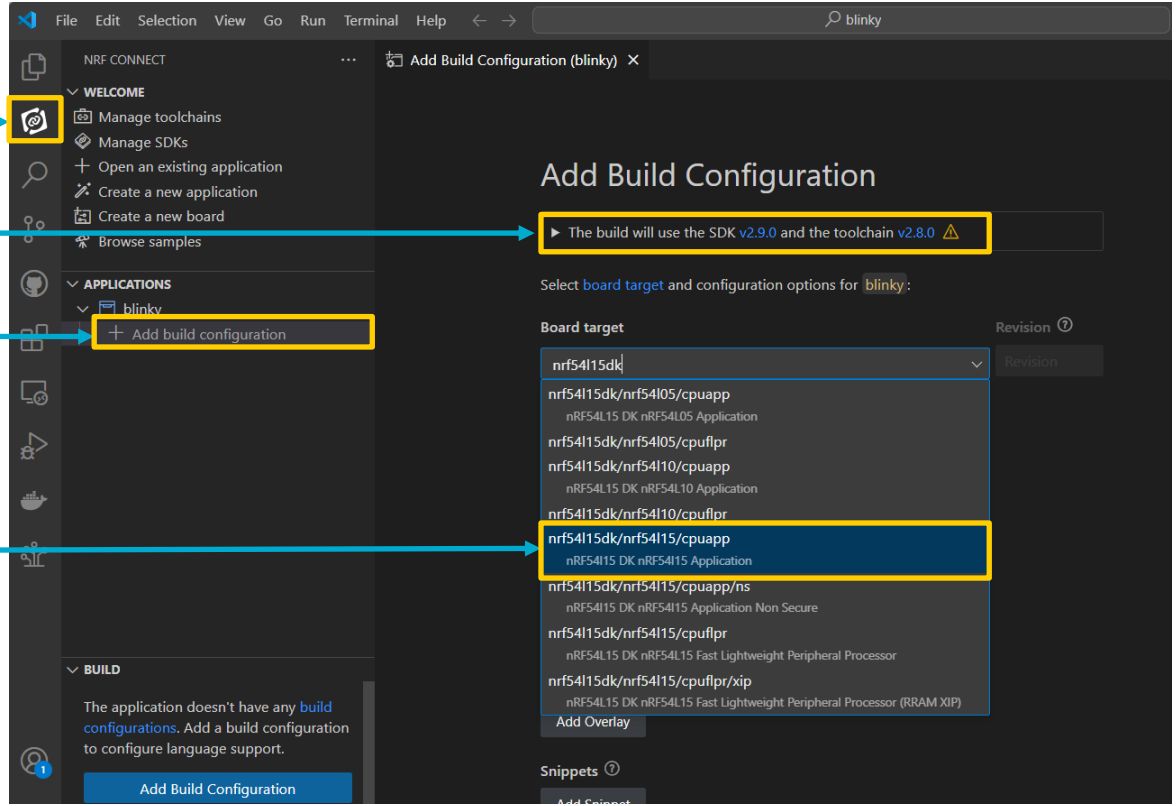
VC has opened the project in the Explorer view



© Nordic Semiconductor

Setting up build configurations 1/3

1. Select the nRF Connect Extension
3. Select the version of the SDK and toolchain
2. Add build configuration
4. Select the target board nRF54l15dk/nrf54l15/cpuapp
   - Board nRF54L15DK
   - SoC nRF54L15
   - Core cpuapp



© Nordic Semiconductor

Board Terminology Diagram

bl5340_dvk@1.2.0/nrf5340/cpuapp/ns

Board name | Board revision | SoC | CPU cluster | Variant

Board qualifiers

Board target

List all board names: `west boards`

List Nordic board names: `west boards -n nrf`

Or webpage:

https://docs.nordicsemi.com/bundle/ncs-latest/page/zephyr/boards/index.html

https://docs.zephyrproject.org/latest/boards/index.html#

board.yml - Notepad

File Edit Format View Help

```
board:
  name: bl5340_dvk
  vendor: lairdconnect
  socs:
  - name: 'nrf5340'
    variants:
    - name: 'ns'
      cpucluster: 'cpuapp'
```

Select Kconfig configuration if empty the **default** is used

Select device tree overlay if empty the **default** is used

Add code snippet common build code

Give a name for the current build directory

Use **Build system default** or **Use sysbuild**

Kconfig overlay added to base Kconfig

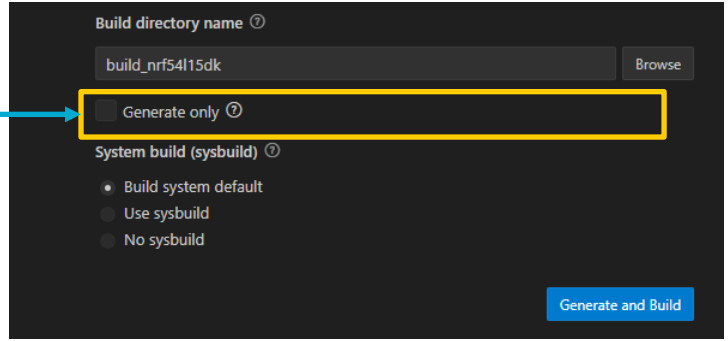
Additional devicetree overlay added to base devicetree

Select **Optimize for debugging**

Add Kconfig fragments and extra Cmake arguments if needed

Setting up build configurations 3/3

Keep box unchecked to generate the configuration and directly build the project.



Finally, hit Generate and Build

Tip: Press CTRL+J and select the "Terminal" tab page to view to progress of the building of the configuration

© Nordic Semiconductor

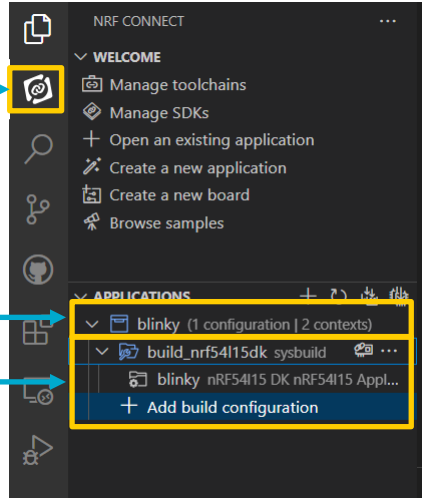
Build completed 1/2

Select the **nRF Connect** extension

Refers to the **“blinky”** project in the **VC Workspace**

Show the available **build configurations** for this application.

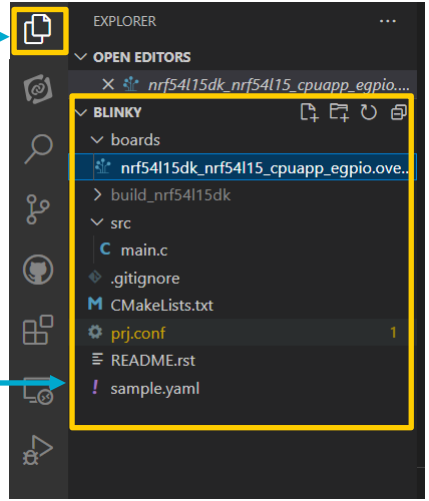
Multiple different builds are possible for a application. For example a debug and a production build.



Build completed 2/2

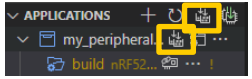
Select the **File explorer** extension

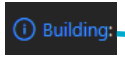
Show the available files in with in the application project.

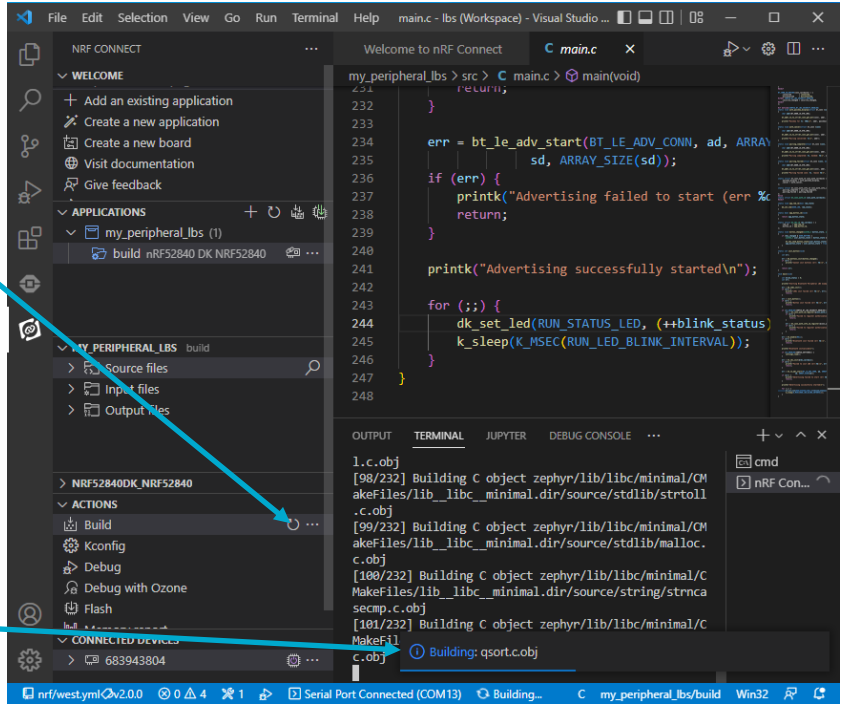


© Nordic Semiconductor

Building your first app

- To (re)build you app click “build” in the Actions view
  - Click here to force a pristine build (a forced clean build)
- If you have several applications or build configurations, you can build all with one click: 

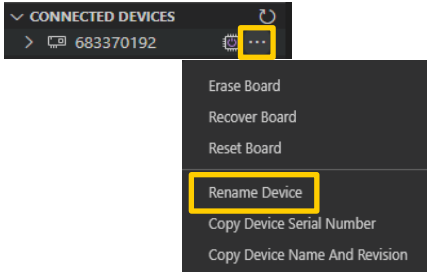
- Building progress can be viewed by clicking 

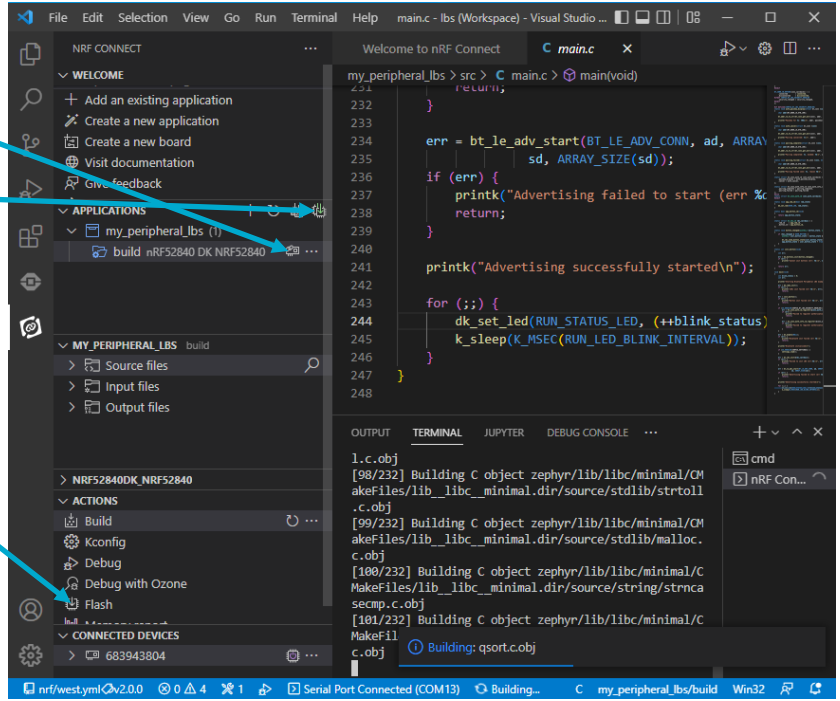


© Nordic Semiconductor

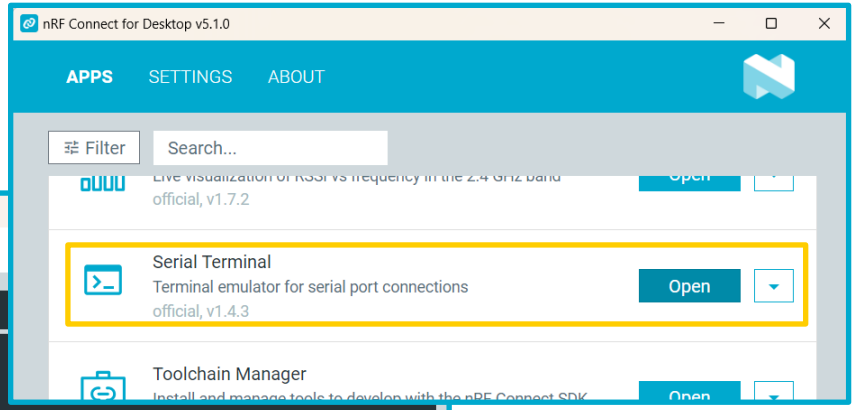
Flashing connected boards

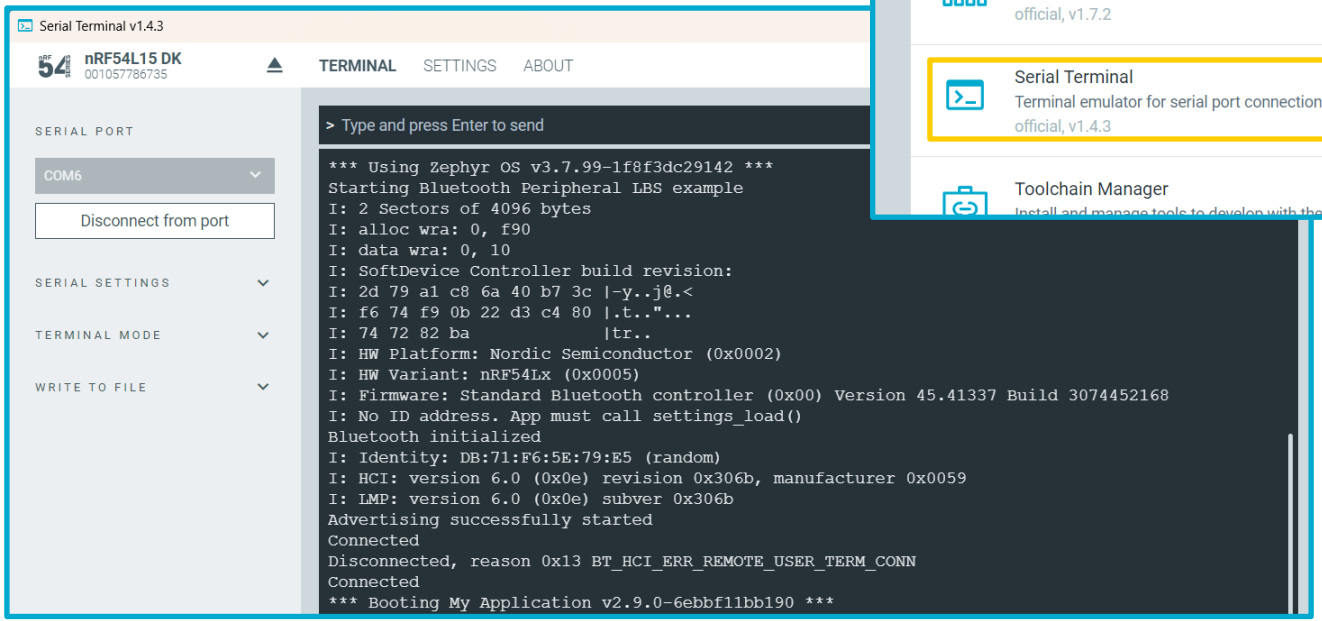
- Link your build config to a connected board
- And flash all linked boards with one click
- Flash your active build
- Give your board a name:





nRF Connect For Desktop
Serial Terminal

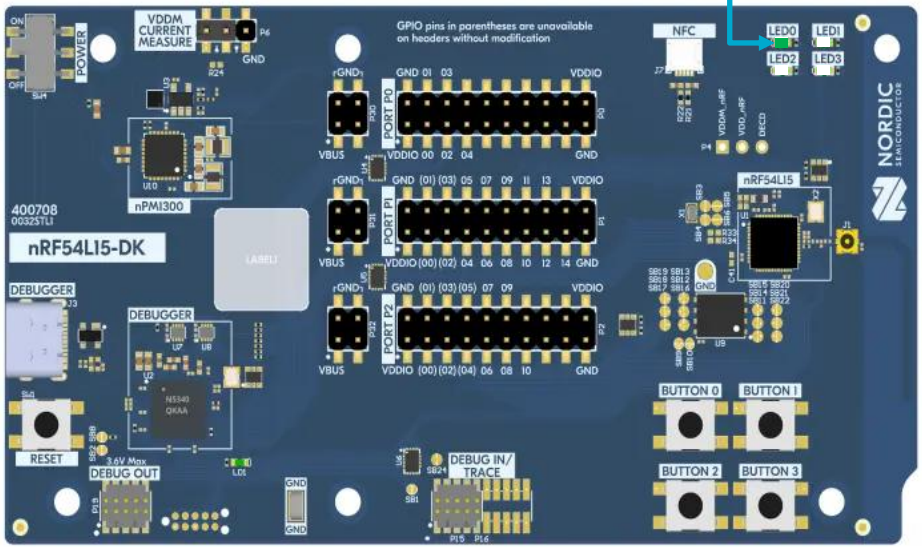




© Nordic Semiconductor

The blinky in action

nRF54L15DK with LED0 blinking



blinky Project Review

© Nordic Semiconductor

Zephyr Sample: Blinky

In the following slides, we will examine the blinky sample line by line to understand how this program works.

The blinky sample comes as part of the nRF Connect SDK and is found at this location:
<ncs>\zephyr\samples\basic\blinky

```c
#include <stdio.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

/* 1000 msec = 1 sec */
#define SLEEP_TIME_MS   1000

/* The devicetree node identifier for the "led0" alias. */
#define LED0_NODE DT_ALIAS(led0)

static const struct gpio_dt_spec led = GPIO_DT_SPEC_GET(LED0_NODE, gpio

int main(void)
{
    int ret;
    bool led_state = true;

    if (!gpio_is_ready_dt(&led)) {
        return 0;
    }

    ret = gpio_pin_configure_dt(&led, GPIO_OUTPUT_ACTIVE);
    if (ret < 0) {
        return 0;
    }

    while (1) {
        ret = gpio_pin_toggle_dt(&led);
        if (ret < 0) {
            return 0;
        }
        led_state = !led_state;
        printf("LED state: %s\n", led_state ? "ON" : "OFF");
        k_msleep(CONFIG_BLINKY_TIME_ON);
    }
    return 0;
}
```

© Nordic Semiconductor

Include Modules

The blinky sample uses the following modules of the nRF Connect SDK/Zephyr:

- C Standard library **<stdio.h>** for IO, printf
- Kernel services **<zephyr/kernel.h>** for the sleep function **k_msleep()** or kernel **printk()**
- The generic GPIO interface **<drivers/gpio.h>** for the structure **gpio_dt_spec**, the macros **GPIO_DT_SPEC_GET()**, and the functions **gpio_pin_configure_dt()** and **gpio_pin_toggle_dt()**.

The modules are included through the following include lines:

```
#include <stdio.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>
```

Define the Node Identifier

- The line below uses the devicetree macro **DT_ALIAS()** to get the node identifier symbol **LED0_NODE**, which will represent **node led_0**.

```
/* The devicetree node identifier for the
   "led0" alias. Defined in the .dts file */
#define LED0_NODE DT_ALIAS(led0)
```

- The **led_0** node is defined in the devicetree of the nRF54L15-DK. **LED0_NODE** is now a source code symbol that represents the hardware for **LED0**.

NOTE:

There are many ways to retrieve the node identifier. The macros **DT_PATH(), DT_NODELABEL(), DT_ALIAS(),** and **DT_INST().** All return the node identifier, based on different input parameters.

- The **DT_ALIAS()** macro gets the node identifier from the node's alias name, which as we saw in the Devicetree section, is **led0**.

Retrieve Device pointer, Pin nb and config flags

- The macro call **GPIO_DT_SPEC_GET()** returns the structure **gpio_dt_spec led**, which contains the device pointer for node **led_0** as well as the pin number and associated configuration flags.
- The node identifier **LED0_NODE** has this information embedded inside its **gpios** property.
- The second parameter **gpios**, the name of the property containing all this information.

```
static const struct gpio_dt_spec led =
            GPIO_DT_SPEC_GET(LED0_NODE, gpios);
```

Verify that device is ready to use

Now we must pass the device pointer of the device, in this case the **led**, to **gpio_is_ready_dt()**, to verify that the device is ready for use.

```
int ret;
bool led_state = true;

if (!gpio_is_ready_dt(&led)) {
    return 0;
}
```

© Nordic Semiconductor

Configure GPIO Pin

The generic GPIO API function **gpio_pin_configure_dt()** is used to configure the GPIO pin

The led pin has an output (**active high**) and initializes it to a logic 1.

```
ret = gpio_pin_configure_dt(&led,
                            GPIO_OUTPUT_ACTIVE);
if (ret < 0) {
    return;
}
```

Continuously toggle the GPIO Pin

Finally, the blinky main function will enter an infinite loop where we continuously toggle the GPIO pin using gpio_pin_toggle_dt().

In every iteration, we are calling the kernel service function k_msleep(), which puts the main function to sleep for 1 second

```
while (1) {
    ret = gpio_pin_toggle_dt(&led);
    if (ret < 0) {
        return;
    }
    k_msleep(SLEEP_TIME_MS);
}
```

Update our Blinky

DeviceTree: Change the LED Pin

© Nordic Semiconductor

DeviceTree – What is it?

- Defining Hardware Usage in separate files (DeviceTree file)
- These Symbols are then used in your Code
- Advantage:
  - Can easily be reused (Hardware can then adjusted by just changing the DeviceTree file)
  - Perfect for Device Driver
  - … but also usefull for your own Code
  - No Memory overhead due to macros

```
#include <devicetree.h>

#define LED0_NODE DT_ALIAS(led0)

#define LED0 DT_GPIO_LABEL(LED0_NODE, gpios)
#define PIN DT_GPIO_PIN(LED0_NODE, gpios)
#define FLAGS DT_GPIO_FLAGS(LED0_NODE, gpios)

void main(void){
   const struct device *dev;
   dev = device_get_binding(LED0);
   ret = gpio_pin_configure(dev, PIN,
                 GPIO_OUTPUT_ACTIVE | FLAGS);
   ...
}
```

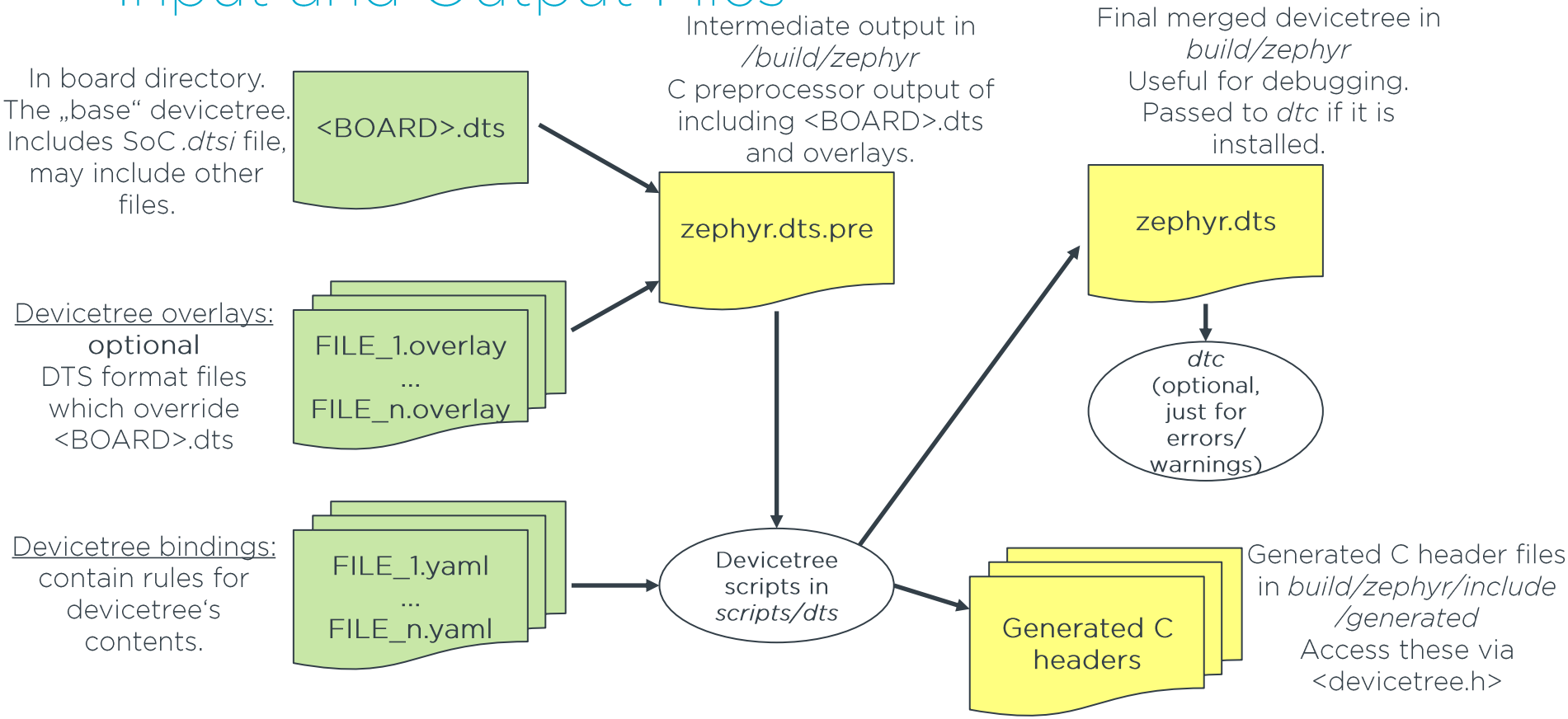
main.c

```
/ {
 leds {
  compatible = "gpio-leds";
  led0: led_0 {
  gpios = <&gpio0 13 GPIO_A
 };
```

DeviceTree Files

© Nordic Semiconductor

Input and Output Files



© Nordic Semiconductor

DeviceTree Structure

Root node

Node (child of root node)

Sub-Node (child of *leds* node)

Node label (e.g. *led3*)

Porperty (e.g. *gpios*)

nrf54l15dk_common.dtsi

C: > Nordic > NCS > v3.2.1 > zephyr > boards > nordic > nrf54l15dk > nrf54l15dk_common.dtsi > / > leds

```
7   #include "nrf54l15dk_nrf54l_05_10_15-pinctrl.dtsi"
8
9   / {
10      leds {
11          compatible = "gpio-leds";
12
13          led0: led_0 {
14              gpios = <&gpio2 9 GPIO_ACTIVE_HIGH>;
15              label = "Green LED 0";
16          };
17
18          led1: led_1 {
19              gpios = <&gpio1 10 GPIO_ACTIVE_HIGH>;
20              label = "Green LED 1";
21          };
22
23          led2: led_2 {
24              gpios = <&gpio2 7 GPIO_ACTIVE_HIGH>;
25              label = "Green LED 2";
26          };
27
28          led3: led_3 {
29              gpios = <&gpio1 14 GPIO_ACTIVE_HIGH>;
30              label = "Green LED 3";
31          };
32      };
33
34      pwmleds {
```

© Nordic Semiconductor

DeviceTree Handling in blinky

❷ Get the Property „gpios“

❶ Point into DeviceTree Node

❸ Use DeviceTree settings to configure GPIO

DeviceTree File:

```
/ {
    leds {
        compatible = "gpio-leds";

        led0: led_0 {
            gpios = <&gpio2 9 GPIO_ACTIVE_HIGH>;
            label = "Green LED 0";
        };
```

```
    aliases {
        led0 = &led0;
```

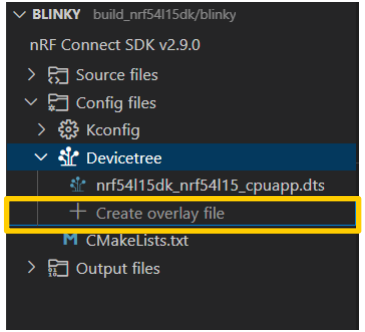
C Code:

```
/* The devicetree node identifier for the "led0" alias. */
#define LED0_NODE DT_ALIAS(led0) ❶

/*
 * A build error on this line means your board is unsupported.
 * See the sample documentation for information on how to fix this.
 */
static const struct gpio_dt_spec led = GPIO_DT_SPEC_GET(LED0_NODE, gpios); ❷

    ret = gpio_pin_configure_dt(&led, GPIO_OUTPUT_ACTIVE); ❸
    if (ret < 0) {
        return 0;
```

© Nordic Semiconductor

Change the LED using devicetree overlays

- First step: Create an overlay file
- When the file name is identical to the specified board in the “build configuration” it will be used automatically
- When using the nRF54L15DK the file name is **“nrf54l15dk_nrf54l15_cpuapp.overlay”**
- When another name is used, it must be added tothe build configuration as a **“Extra devicetree overlay”** file
- Use VC to create an overlay file
- TIP: Skip the pristine build & first perform the changes in the overlay file, then do a pristine build!



Option A: overwrite led0 node in overlay file

- Blinky uses the "led0" node referenced in main.c

```
/* The devicetree node identifier for the "led0" alias. */
#define LED0_NODE DT_ALIAS(led0)
```

- Overwrite led0 to a different pin (here onboard LED2, add to overlay:

```
&led0 {
    gpios = <&gpio2 7 GPIO_ACTIVE_HIGH>;
};
```

```
nRF54L15-DK – HW Setup:
        LED0 = Port 2, Pin 9
        LED1 = Port 1, Pin 10
        LED2 = Port 2, Pin 7
        LED3 = Port 1, Pin 14
```

© Nordic Semiconductor

Option B: Create a new DTS node group

- Create the DTS node group "board_leds" and add the node my_led1 with its node label my_led_1
- Specify the I/O of the LED output

```
/ {
    board_leds {
        compatible = "gpio-leds";
        my_led_1: my_led1 {
            gpios = <&gpio1 10 (GPIO_ACTIVE_HIGH)>;
            label = "My Green LED 1";
        };
    };
};
```

- Change devicetree node identifier in main.c to new node definition (my_led1). We now use the DT_PATH to identify the node. DeviceTree path: /board_leds/my_led1

```
/* The devicetree node identifier for the "led0" alias. */
#define LED0_NODE DT_PATH(board_leds, my_led1)
```

Option C: Refer with NODELABEL

- The nRF54L15DK board target defines already LED1. Let's use this one for blinking.

```
28        led3: led_3 {
29            gpios = <&gpio1 14 GPIO_ACTIVE_HIGH>;
30            label = "Green LED 3";
31        };
```

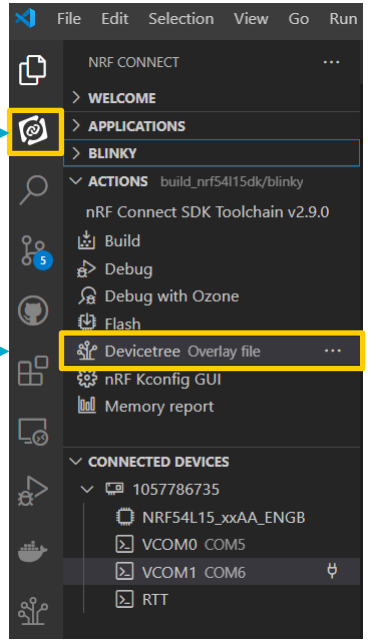
- Change in main.c the node identifier and use DT_NODELABEL to point to LED3 instead of LED0.

```
/* The devicetree node label for LED3 is "led3". */
#define LED0_NODE DT_NODELABEL(led3)
```

Open visual devicetree editor

Select the nRF Connect Extension

In the actions section Select “Devicetree” to open the visual editor



Modifies directly the devicetree files.

Be carefull that changes could effect not only this application. Some changes can be written into the board files.

© Nordic Semiconductor

File Edit Selection View Go Run Terminal Help

blinky

DEVICETREE VISUAL EDITOR

BUILD CONTEXTS
CONTEXT FILES
CONTEXT OVERVIEW
NODES blinky: build_nrf54l15dk/bl...
- aliases
- board_leds
- buttons
- chosen
- clocks
- cpus
- leds
  - led0
  - led1 1
  - led2
  - led3
- pinctrl
- psa_rng
- pwmleds
- rng_hci
- soc

nrf54l15dk_nrf54l15_cpuapp.overlay U | nrf54l15dk_nrf54l15_cpuapp.overlay U | prj.conf M | main.c 1, M

nrf54l15dk_nrf54l15_cpuapp.overlay

led_0

PROPERTIES
label
Green LED 0

PINS
GPIOS
P2.09
Assign...

ADVANCED
No properties to show.

LABELS
led0

DESCRIPTION
GPIO LED child node

Push button 0 KeyCode: 11
Push button 1 KeyCode: 2
Push button 2 KeyCode: 3
Push button 3 KeyCode: 4
Green LED 0
Green LED 1
Green LED 2
Green LED 3
My Green LED 1

P1.13 P1.09 P1.08 P0.04 P2.09 P1.10 P2.07 P1.14 P1.14

188 KiB

buttons: button0 button1 button2 button3
leds: led0 led1 led2 led3
board_leds: my_led_1
GPIOs

pwm20
pwm_led1 CH0 CH1 CH2 CH3
OUT0 OUT1 OUT2 OUT3
P1.10
sw-pwm

Time
timer 0 1 2 3 4 5 6
wdt 0 1
clock

Configuration
ficr
uicr

Signalling
dppic 0 1 2 3
egu 0 1
gpiote 0 1

Power
power
regulators

Communication
nfct

Other
comp
temp

master* 0 10 2 0

blinky: build_nrf54l15dk/blinky

Update our Blinky

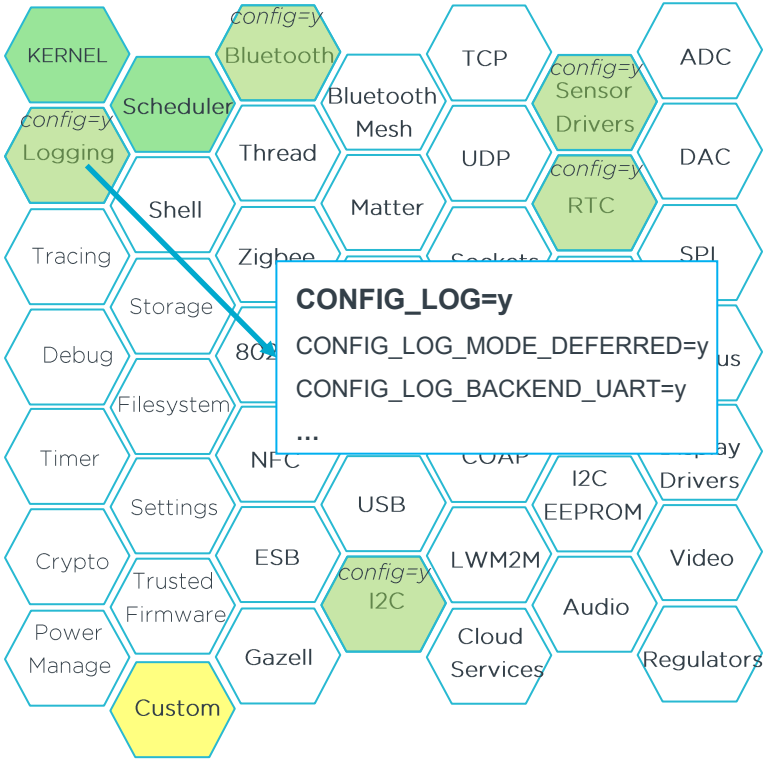
KCONFIG: Use Zephyr Logging

© Nordic Semiconductor

Configuration System (KCONFIG)

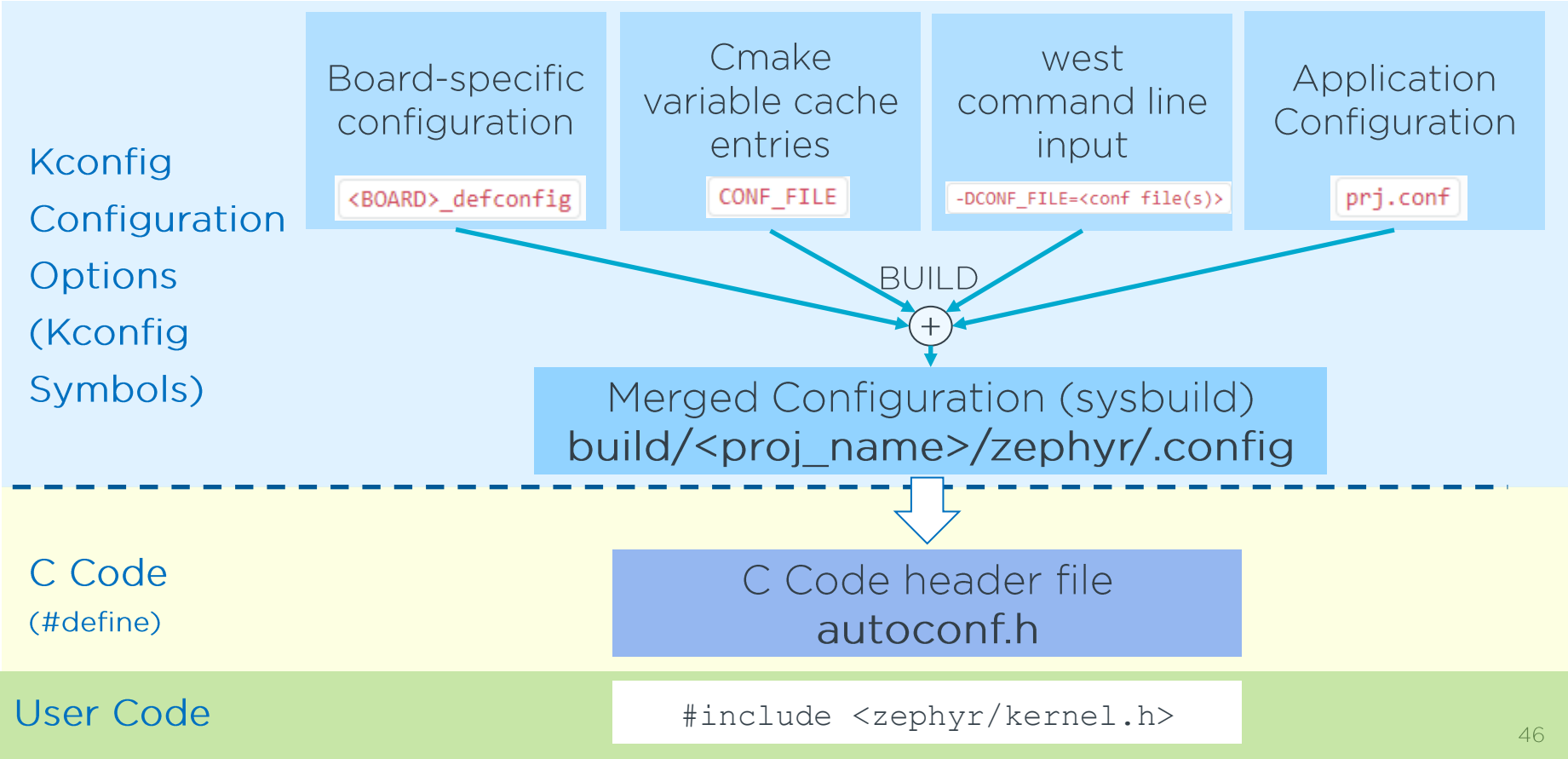
- Zephyr Kernel and Sub-System can be configure at build time
  - Add software modules to your project
  - If a software module was added, further CONFIG symbols appear and allow you to configure the module
- Custom Kconfig is possible

- <u>Goal</u>: Configure software features without changing source code



© Nordic Semiconductor

Configuration for an Application



© Nordic Semiconductor

Hands-on: Adding Zephyr Logging

1) Enable Logging by adding following line in *prj.conf* file:

```
# Enable Logging

CONFIG_LOG=y
```

2) Register Logging in your Software Module (in our case it is main.c file):

```
#include <zephyr/logging/log.h>

/* LOG MODULE REGISTRATION */

LOG_MODULE_REGISTER(MyApp,LOG_LEVEL_INF);
```

3) User it by adding message that should be sent:

```
LOG_INF("LED state: %s", led_state ? "ON" : "OFF");
```

Update our Blinky

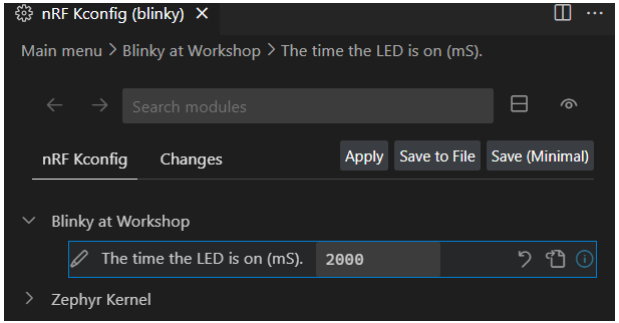
Adding kernel definitions

© Nordic Semiconductor

Build a kernel configuration menu

- Insert a new file in the root of the "**blinky**" project with the name "**Kconfig**"
- This defines **CONFIG_BLINKY_ON_TIME** with a default value of "**1000**"
- Select the menu "nRF Kconfig GUI" in the "ACTIONS" section in the nRF Connect Ext.

```
menu "Blinky at Workshop"
config BLINKY_TIME_ON
    int "The time the LED is on (mS)."
    default 1000
endmenu

menu "Zephyr Kernel"
source "Kconfig.zephyr"
endmenu
```



© Nordic Semiconductor

Add to application: edit manually

- Within our "prj.conf" we can modify the value CONFIG_BLINKY_ON_TIME
- Add the following into the "prj.conf" file in your blinky project.

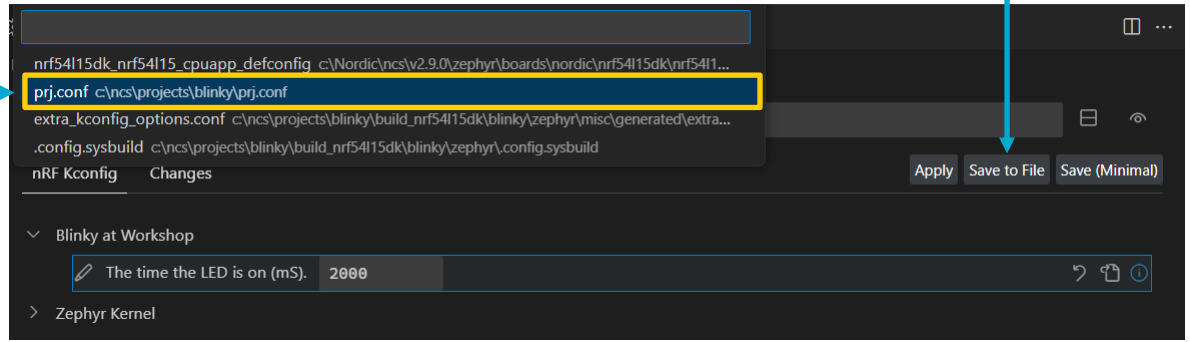
```
CONFIG_BLINKY_TIME_ON=2000
```

- Now replace in our main.c the wait (while(1)) loop with this definition:

```
k_msleep(CONFIG_BLINKY_TIME_ON);
```

Add to application: nRF Kconfig GUI

Select Save to file to make the changes to a Kconfig file



Select Kconfig file to make modify the changed values.
Use the prj.conf file of the current application.

Selecting the nrf54l15dk_nrf54l15_cpuapp_defconfig will modify the SDK for all applications.
When written in extra_kconfig_options.conf the modification is removed with pristine build